

RAG Features and Why They Matter

Adaptive Learning Project — compiled from team discussion (architecture decision record + follow-up conversation on RAG structure)

This document compiles the RAG-related features discussed so far and the reasoning behind each one. It doesn't introduce anything new — it's a record of what the team already decided or discussed, organized for reference. Everything here traces back to one root finding from the instructor interviews: trust was named as the #1 adoption blocker, and instructors gave specific, testable conditions for it — cite sources, show accuracy, avoid hallucination. The features below are largely the engineering answer to that finding.

1. Core Pipeline Features

Document parsing / ingestion

Converts instructor-uploaded files (PDFs, possibly DOCX) into usable text. Scoped to text-based PDFs for now; scanned/image-only pages would need OCR, which was explicitly flagged as out of scope rather than left as a silent failure case.

Chunking strategy

Splits uploaded content into retrievable pieces. Starting point is fixed-size chunking with overlap — simple and fast to implement. Structure-aware chunking (splitting on headings/paragraphs first) was noted as a likely quality improvement for educational content specifically, since a student's question usually maps to a concept, not an arbitrary token window — but only worth the extra effort if testing shows plain chunking isn't good enough.

Embedding model

Turns text into vectors for similarity search. Must be open-source per the team's no-paid-budget constraint. Specific model choice was left open, with a note to check a current embedding benchmark rather than picking by name recognition, since rankings shift over time.

Vector store

Where embeddings are stored and queried. Chroma was recommended as the default specifically because it's quick to set up for a prototype on a short timeline; FAISS, Qdrant, and Weaviate were discussed as alternatives with more setup overhead but more built-in features.

Retrieval strategy (naive to agentic)

How relevant chunks get found. Naive top-k similarity search was the recommended starting point. Hybrid search (adding keyword/BM25 matching) was flagged as a likely upgrade if exact terminology matters and naive search misses it. Multi-step agentic retrieval (the model issuing multiple retrieval queries and reasoning across them) was scoped as a stretch goal, specifically relevant to “connects dots across topics” and “more depth” requests from the feature backlog, not a day-one requirement.

Metadata filtering / multi-tenancy

Scopes retrieval so a student in one course can't pull content from another course/institute's uploads. Flagged as something to build into the data model from day one, since retrofitting isolation later is painful.

2. Trust & Credibility Features

This is the section most directly tied to the interview evidence. Instructors named these conditions explicitly — e.g. wanting fact-checking and references, expecting hallucination and wanting a way to catch it, and in one case (a respondent drawing on a clinical/medical framing) wanting validation “in a prospective randomized study” before trusting an AI tutor at all.

Citation / grounding mechanism

Every answer traces back to the specific source passage it came from, shown to the user. This is a data-plumbing requirement through the whole pipeline (chunk and document metadata has to survive from retrieval through to the UI), not just a display feature. Directly answers instructors' named condition of wanting references/fact-checking.

Graceful “not found” handling

A similarity-score threshold below which the system says it doesn't have information on a topic in the uploaded materials, instead of generating an answer anyway. This is the concrete implementation of the hallucination-handling condition instructors named.

Self-RAG-inspired groundedness self-check

A lighter, promptable approximation of the Self-RAG research idea: rather than replicating the full technique (which likely requires a specially trained model), the team discussed two promptable pieces — a retrieval-necessity check (does this question even need a lookup?) and a post-hoc check where the model verifies its own draft answer against the retrieved passage before finalizing it.

Live per-answer groundedness signal

Discussed as an upgrade beyond an offline evaluation report: showing a real-time indicator per answer (e.g. “grounded” vs. “couldn't fully verify against your materials”) rather than only a pre-demo benchmark number. Flagged as a stronger credibility play because judges and instructors can watch the system self-assess live, rather than take the team's word for an offline claim. Trade-off noted: an extra model call per answer, plus the scorer itself can be wrong — not a free upgrade, but one the team leaned toward prioritizing given it was named as a major selling point.

3. Evaluation & Scoring

Offline evaluation set

A curated set of test questions with known correct answers/source passages, used to measure and improve retrieval and generation quality before the demo. RAGAS was named as an open-source framework built specifically for this kind of RAG evaluation (covering things like whether an answer is faithful to retrieved context, and whether the right context was retrieved). A lighter alternative discussed: hand-build 10–20 questions and grade manually if standing up a full evaluation framework is more than the timeline allows. Produces a concrete artifact — e.g. “X% of test answers were fully grounded in cited material” — that

directly answers instructors' stated need for evidence of accuracy before trusting the tool (echoed by one respondent saying "I will test it first").

Live groundedness display

See Section 2 — discussed as the stronger, demo-visible complement to the offline eval set rather than a separate feature.

4. Personalization Retrieval

Architecturally distinct from content-RAG (which retrieves course material to answer questions). This is about retrieval-style mechanisms over the student's own data to drive pacing and format adaptation. The discussion drew a line between two readings of this idea:

Structured learner-profile queries

Most personalization data (weak areas, preferred explanation format, per-topic mastery level) is structured, not free text — so this is more naturally a plain database query than a semantic vector search. Noted as the likely right default rather than standing up a second vector store.

Semantic retrieval over interaction history

Would only be needed if the team is storing full free-text transcripts of past sessions and wants to search them by meaning. Flagged as extra complexity to add only if a plain profile lookup proves insufficient.

External system integration (e.g. an LMS)

A different and much larger scope addition — real API integrations, a new privacy/security surface entirely. Flagged as out of scope for now unless there's a specific reason to bring it in.

Note: which reading of "personalization retrieval" the team actually intends was raised as an open question in discussion and hadn't been confirmed as of this writing.

5. Discussed but Flagged as a Caution

Web search / scraping as a fallback for more depth

Raised as an option for when uploaded content doesn't have enough depth to answer a question. The team's own feature list already specifies that "more depth" should stay aligned to uploaded/curriculum content — pulling from the open web directly conflicts with that, since a web page was never vetted by the instructor the way uploaded material was. This reintroduces the exact unvetted-source risk the trust features above are built to avoid. Practical concerns were also raised: real-time web search generally isn't free at real usage volume, and scraping has its own reliability and terms-of-service complications. If built at all, the recommendation was to keep it as a separate, clearly-labeled, opt-in mode — not blended into the "grounded in your instructor's materials" trust story — and to treat it as a stretch item, not core.